

MVP

MVP User Flow

This is the flow the semester is built around. Every task below exists to support one of these steps.

Account



Clarify Goals



Create Split



Start Workout



Log Sets / RIR



Save Workout



View Machine-Specific History



View Progression

Step by step:

1. **Account** — User signs up / logs in. Auth handled by Supabase.

2. **Clarify Goals** — User selects a training goal (hypertrophy, strength, or general). This determines how the progression engine weights its suggestions later. User also sets their gym(s).
3. **Create Split** — User picks or builds a split (Upper/Lower, PPL, Full Body, Bro Split, Arnold, Freestyle), names the days, and adds exercises to each day. Per exercise, the user sets their own rep range and which machine/equipment it uses.
4. **Start Workout** — App shows the next day in the split rotation, pre-loaded with that day's exercises and the user's last recorded weight/reps for each.
5. **Log Sets / RIR** — For each set: weight, reps, and effort (RIR). This is the highest-traffic screen in the app and needs to be fast to use at the gym.
6. **Save Workout** — Session is written to the database, dated, and tied to the split day. Estimated 1RM is computed per set at write time.
7. **View Machine-Specific History** — For any exercise, show every set ever logged on that specific machine, in date order.
8. **View Progression** — Trend view per exercise and per muscle group: weight, reps, and estimated 1RM over time.

MVP Features

Authentication / Account

- Create account
- Log in
- Store training goal
- Store gym(s)

Splits

- Choose split type
- Create split days
- Name split days
- Add exercises to a day
- Reorder exercises within a day
- Set rep range per exercise
- Assign machine/equipment per exercise
- Edit / delete split days and exercises

Workout Logging

- Start a workout from the next day in rotation

- Show last session's numbers per exercise
- Log weight, reps, RIR per set
- Add / remove sets mid-workout
- Save workout
- Backfill a workout for a past date

History

- View all sets for a given exercise
- Filter history by machine
- View a past workout session

Progression

- Estimated 1RM per set (computed and stored at write time)
- Trend chart per exercise over time
- Trend view per muscle group

Stretch Goal — Progression Feedback Engine

- Suggest next session's weight and reps based on logged RIR, staying inside the user's rep range
- Weight suggestions differently depending on the user's stated goal
- Flag when a muscle group needs more recovery time based on recent high-effort sets

Core Data

Schema lives in supabase/migrations/. Full ERD in docs/erd.md.

users

- id, email, display_name, main_gym_id, training_goal, created_at

gyms

- id, user_id, name

movements *(abstract movement, for grouping only)*

- id, name, muscle_group

exercises (*a movement on specific equipment — this is the per-machine unit*)

- id, user_id, movement_id, gym_id, machine_label, rep_range_min, rep_range_max, created_at

splits

- id, user_id, name, split_type, program_start_date

split_days

- id, split_id, name, order_index

split_day_exercises

- id, split_day_id, exercise_id, order_index

workout_logs

- id, user_id, split_day_id, log_date, week_number, notes

sets

- id, workout_log_id, exercise_id, set_number, weight, reps, effort_rating, estimated_1rm

bodyweight_logs

- id, user_id, log_date, weight

Key points:

- exercises is the per-machine unit. Rep range lives here, not on movements.
 - sets carries the training data. estimated_1rm is computed on write, not derived on read — the progression views query it constantly.
 - movements is optional grouping so the user can see "all my leg press variants" without merging their separate histories.
-

Tech Stack

- **Client:** React Native (Expo), TypeScript — iOS and Android primary, web secondary
- **Data layer:** Supabase (PostgreSQL) — auth, storage, row-level security, auto-generated REST API
- **Algorithm service:** Python (FastAPI) — progression logic only
- **Repo:** GitHub monorepo

How the pieces talk to each other

React Native app

└──→ Supabase directly (auth, CRUD: splits, exercises, sets, history)

└──→ FastAPI (progression suggestions, 1RM calculations)

↓

Supabase (reads the same tables)

Important: the app talks to Supabase *directly* for ordinary reads and writes. Supabase already generates a REST API, so we do not write CRUD endpoints for logging a set or fetching history. FastAPI exists only for logic Supabase cannot express — the RIR progression algorithm and 1RM math. Do not route basic CRUD through FastAPI.

Repo Structure

IRON-Workout-App/

└── app/ React Native (Expo)

└── api/ FastAPI service

└── supabase/ schema SQL, migrations, RLS policies

└── docs/ this doc, ERD, research library

└── .env.example

└── CONTRIBUTING.md

└── README.md

Ownership

Work is divided by system so we aren't editing the same files. Everyone contributes across the stack when needed; this is primary ownership.

Olakiite — Research & Progression Logic

- Compile and maintain the peer-reviewed research library (docs/research.md)
- Design the progression algorithm: how RIR maps to a weight/rep suggestion
- Choose and document the 1RM formula (weight-dependent, not fixed-factor classical Epley)
- Implement the algorithm as pure, unit-tested functions in api/app/services/progression.py
- Define how training_goal changes the suggestion weighting
- Tune the engine against real logged data late in the semester

Aliyah — Frontend / UI

- Scaffold the Expo app: navigation, screen structure, routing
- Build the auth and onboarding screens (account, goals, gym setup)
- Build the split creator / day builder
- Build the set logging screen — the most-used screen, needs to be fast and thumb-friendly
- Build history and progression views, including charts
- Theming system
- UI polish, device testing, TestFlight / Play internal build

Nathaniel — Backend / Data Infrastructure

- Write and own the Supabase schema (tables, relationships, foreign keys)
- Set up Supabase auth
- **Enable Row Level Security on every table and write the policies.** RLS is off by default on new tables. Without it, any logged-in user can read anyone's workout data.
- Write SQL migrations into supabase/migrations/ so schema changes are visible in git
- Scaffold the FastAPI service and define its endpoint contracts
- Integration testing across app / Supabase / FastAPI
- Contributor documentation (CONTRIBUTING.md)

Development Strategy

Phase 0 — System Design (do this together, before building features)

Decide and write down:

- Final schema (all tables, all fields, all relationships)
- Which operations go to Supabase directly vs. through FastAPI

- FastAPI endpoint contracts — request and response shapes, written down *before* they're implemented
- RLS policy approach
- Git workflow, branch naming, PR expectations
- What training_goal values exist and what each one changes
- What the effort/RIR scale looks like in the UI and how it's stored
- How split rotation determines "the next day"

Deliverable: everyone can draw the data flow from a tapped set on the logging screen, through Supabase, into the progression engine, and back out as a suggestion.

Phase 1 — Schema and the Pipe

Nathaniel gets the schema live with auth and RLS. Aliyah scaffolds the Expo app and navigation. Olakiite compiles the research and starts designing the algorithm.

Deliverable: a test user can sign up in the app and write one row (a bodyweight log) to Supabase, and read it back. Small on purpose — it proves the whole pipe works before anything complex is built on top of it.

Phase 2 — Core Logging Flow

The bulk of the semester. Splits, day builder, set logging, save, history. App talks to Supabase directly for all of it.

Deliverable: the MVP user flow works end to end, up through machine-specific history.

Phase 3 — Progression Views

Estimated 1RM displayed, trend charts per exercise and muscle group.

Deliverable: a user who has logged several weeks of workouts can see their strength trend.

Phase 4 — Feedback Engine (stretch)

FastAPI progression endpoint wired to the app. Suggestions appear pre-filled on the logging screen.

Deliverable: starting a workout shows a suggested weight and rep target per exercise, with the reasoning traceable to a cited study.

Working in Parallel

The dependency chain is: schema → endpoints → UI. Left alone, that means Aliyah waits on Nathaniel, who waits on the schema. Two rules prevent that:

1. **Contracts before code.** Nathaniel writes down what an endpoint will accept and return before implementing it. Aliyah and Olakiite build against that contract with mock data, then swap in real calls when the endpoint lands.
2. **Schema changes get flagged before merging.** Once the schema is live, a renamed column silently breaks someone else's work. Raise schema changes in the weekly sync, and put them in supabase/migrations/ so they're visible in git.

Git Workflow

- main stays always-deployable. Nothing merges that doesn't build.
- Branch per feature, named by what it is: feature/day-builder-ui, feature/supabase-schema, feature/progression-endpoint.
- Merge to main via pull request, never direct push.
- One other person skims every PR before merge — mainly to catch "this breaks my part," not to gatekeep style.
- Commit early, commit often. RCOS evaluates on contribution history, so incremental commits are better than one large drop.
- Open a GitHub Issue per task and attach it to a Milestone matching the proposal dates.

Never Commit

- .env files
- Supabase service_role key (this key bypasses RLS — it must never reach the client or the repo)
- Database password

The anon/public Supabase key is safe in client code; RLS is what protects the data.

First Major Milestone

Do not build charts or the feedback engine until this works:

Create account → set goal → create a split with one exercise

→ log one set with weight/reps/RIR → save → see that set in history

This proves auth, RLS, the schema, the write path, and the read path all work together. Everything after it is the same pattern repeated.

Out of Scope This Semester

Documented so they don't creep in:

- Food / calorie tracking

- Form reviewer (video-based technique feedback)
- Shared gym or machine data across users
- Social features, friends, leaderboards
- Velocity-based training (requires bar-speed hardware)
- Apple Health / Google Fit integration
- Offline mode with sync conflict resolution

The priority is proving the core flow: **account** → **split** → **log** → **history** → **progression**.

